Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Übung 03: Einführung in VHDL

M.Sc. E. Trumann, M.Sc. J. Homann, Prof. Dr.-Ing. G. Payá Vayá,
7. November 2025

EDA-Playgrounds

- freiwillig, für zuhause
- kostenlos, man muss sich registrieren
- VHDL-Code schreiben und simulieren
- edaplayground.com
- Full-Adder Beispiel: https://www.edaplayground.com/x/2vmt

VHDL Git-Übungen

- Übungsblätter jede Woche neu
- Ein Git-Repository für jeden Teilnehmer (wenn nicht E-Mail an uns)
- Templates im Git-Repository und EDA Playground (QR Codes)
- Eine Gitlab-Continous-Integration überprüft eure Lösungen
- Aufgaben sind sehr ähnlich zur Klausur
- Bearbeitung bevorzugt mit EDA Playground und Gitlab Webinterface

Git-Übungen bearbeiten Step-by-Step

- Siehe separates PDF: Git-Tutorial in Stud.IP

Kurzfragen

a) Was machen folgende VHDL-Ausdrücke und sind diese synthetisierbar? Wenn ja, wie könnte die dazugehörige Hardware aussehen?

```
-- A und B sind jeweils 8 Bit lange Vektoren (std_ulogic_vector)
A <= std_ulogic_vector(shift_left(unsigned(B),3));
```

Kurzfragen

a) Was machen folgende VHDL-Ausdrücke und sind diese synthetisierbar? Wenn ja, wie könnte die dazugehörige Hardware aussehen?

```
-- A und B sind jeweils 8 Bit lange Vektoren (std_ulogic_vector)
A <= std_ulogic_vector(shift_left(unsigned(B),3));
```

Der Vektor B wird um 3 nach links geshiftet und das Ergebnis in A gespeichert. Ja, synthetisierbar.
$\rightarrow$ Ein Shift ist in Hardware eine Verdrahtung. Die 5 MSB von A übernehmen ihre Werte direkt von B (5 LSB). Die restlichen Bits von A werden mit Nullen verbunden/gefüllt.

Kurzfragen

b) Welche Bitzustände des Typen *std_ulogic* sind im *ieee.std_logic_1164.all*-Package definiert?

Kurzfragen

b) Welche Bitzustände des Typen *std_ulogic* sind im *ieee.std_logic_1164.all*-Package definiert?

```
TYPE std_logic is
('U', -- Uninitialized
'X', -- 0/1 ? (Konflikt)
'0', -- 0
'1', -- 1
'Z', -- hohe Impedanz
'W', -- Weak, L/H? (Konflikt)
'L', -- schwache 0
     -- (für Pull-Down)
'H', -- schwache 1
     -- (für Pull-Up)
'-'  -- don't care
);
```

Kurzfragen

c) Was ist der Unterschied zwischen *std_logic* und *std_ulogic*?

Kurzfragen

c) Was ist der Unterschied zwischen *std_logic* und *std_ulogic*?

std_logic ist 'resolved'. *std_ulogic* ist 'unresolved'. D.h. wenn ein Signal mehrere Treiber hat, wird das resultierende Signal bei *std_logic* 'resolved'.
Beispiel:

- aus '0' und '1' wird 'X'.
- aus 'Z' und '1' wird '1'.

Bei *std_ulogic* würde ein Fehler kommen, wenn ein Signal mehrere Treiber hat. Oft hat man als Entwickler einen Fehler gemacht, wenn ein Signal mehrere Treiber hat $\rightarrow$ wir verwenden *std_ulogic*

Kurzfragen

d) Was ist der Unterschied zwischen *unsigned*- und *signed*-Vektoren?

Kurzfragen

d) Was ist der Unterschied zwischen *unsigned*- und *signed*-Vektoren?

(Un-)signed bestimmt die Interpretation eines Bitvektors. Unsigned Vektoren werden als vorzeichenlose Zahl interpretiert. Signed Vektoren als vorzeichenbehaftete Zweierkomplement-Zahl.

Zweierkomplement

a) Wie ist die Darstellung der Dezimalzahl 42_{10} als Binärzahl (8-Bit)?

Zweierkomplement

a) Wie ist die Darstellung der Dezimalzahl 42_{10} als Binärzahl (8-Bit)?

$42 = 32 + 8 + 2 = 2^5 + 2^3 + 2^1$
$\rightarrow 00101010$

Zweierkomplement

b) Wie ist die Darstellung der Dezimalzahl -42_{10} im Zweierkomplement (8-Bit)?

Zweierkomplement

b) Wie ist die Darstellung der Dezimalzahl -42_{10} im Zweierkomplement (8-Bit)?

$$-42 = -128 + 86 = -128 + 64 + 16 + 4 + 2 = -2^7 + 2^6 + 2^4 + 2^2 + 2^1$$
$$\rightarrow 11010110$$

Alternative: Positive Binärzahl invertieren und mit 1 addieren:
$00101010 \rightarrow 11010101 \rightarrow 11010110$

Zweierkomplement

c) Wie ist die Darstellung der Dezimalzahl -42_{10} im Zweierkomplement (**10-Bit**)?

Zweierkomplement

c) Wie ist
die Darstellung der Dezimalzahl -42_{10} im Zweierkomplement (**10-Bit**)?

$-42 = -512 + 256 + 128 + 86 =$
$-512 + 256 + 128 + 64 + 16 + 4 + 2 = -2^9 + 2^8 + 2^7 + 2^6 + 2^4 + 2^2 + 2^1$
$\rightarrow$ 1111010110

schneller: negative Zahlen werden mit Einsen aufgefüllt: 11010110 $\rightarrow$ 1111010110

Zweierkomplement

d) Gegeben sei eine n-Bit-breite Binärzahl in Zweierkomplement-Darstellung. Welchen Zahlenbereich deckt diese Zahl ab?

Zweierkomplement

d) Gegeben sei eine n-Bit-breite Binärzahl in Zweierkomplement-Darstellung. Welchen Zahlenbereich deckt diese Zahl ab?

Minimum $\rightarrow -2^{n-1}$
Maximum $\rightarrow 2^{n-1} - 1$

Zweierkomplement

e) Berechnen Sie folgenden Ausdruck durch schriftliches **Addieren** der Binärzahlen: $41 - 42$

Zweierkomplement

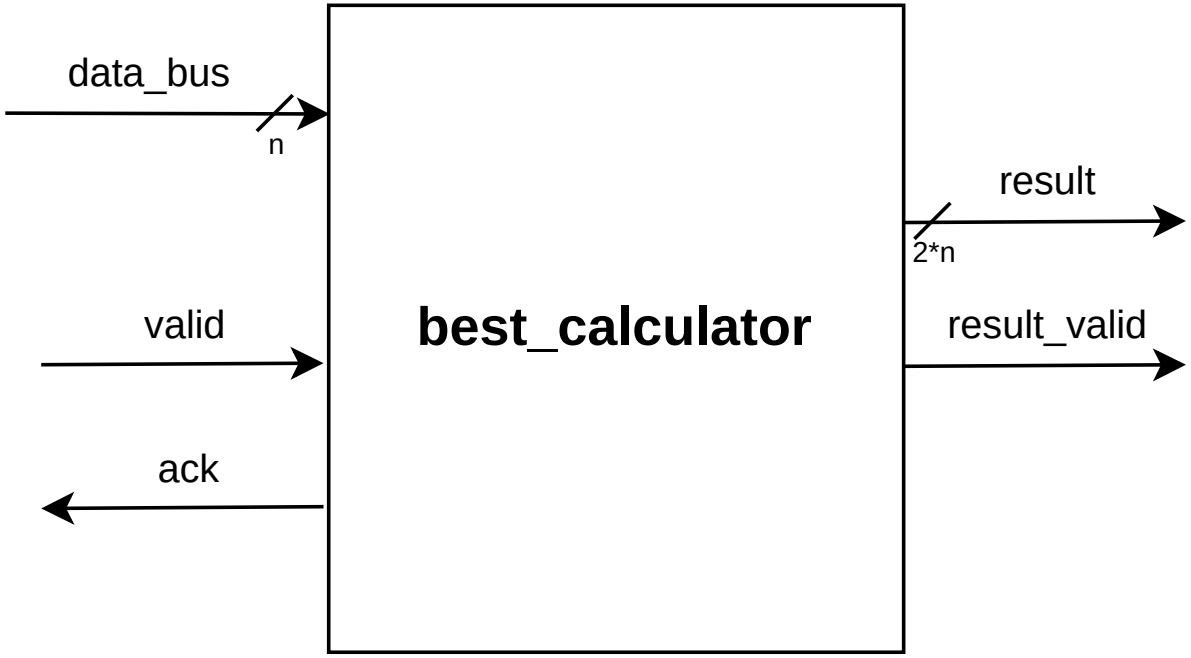
e) Berechnen Sie folgenden Ausdruck durch schriftliches **Addieren** der Binärzahlen (8-Bit): $41 - 42$

$$
\begin{array}{r}
00101001 \\
+\ 11010110 \\
\hline
11111111 \\
\rightarrow -1_{10}
\end{array}
$$

$\rightarrow$ Addition und Multiplikation funktionieren auch im Zweierkomplement

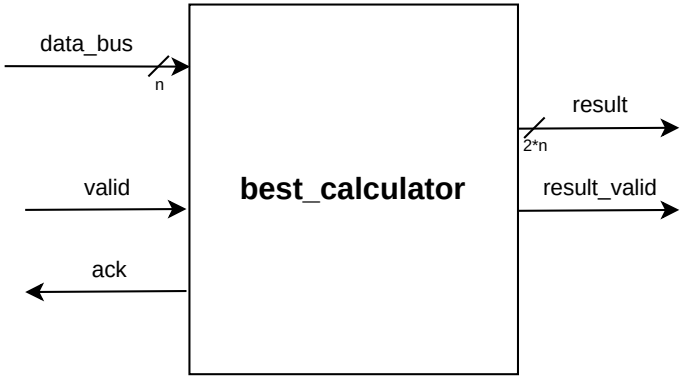
VHDL | Entity

Schreiben Sie die VHDL-*Entity* zu diesem Modul (verwenden Sie *std_ulogic* oder *std_ulogic_vector*):



VHDL | Entity

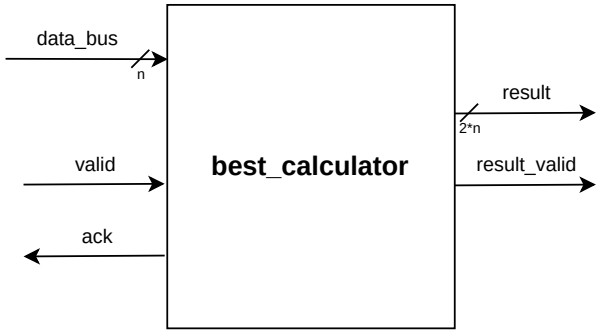
Schreiben Sie die VHDL-*Entity* zu diesem Modul (verwenden Sie *std_ulogic* oder *std_ulogic_vector*):



VHDL | Entity

Schreiben Sie die VHDL-*Entity* zu diesem Modul (verwenden Sie *std_ulogic* oder *std_ulogic_vector*):

```
entity best_calculator is
generic(
  n : integer := 16
);
port(
  valid        : in  std_ulogic;
  ack          : out std_ulogic;
  data_bus     : in  std_ulogic_vector(n-1 downto 0);
  result_valid : out std_ulogic;
  result       : out std_ulogic_vector(2*n-1 downto 0));
end best_calculator;
```



VHDL | Architecture

Gegeben ist folgende *Entity* eines Moduls:

```
entity mir_faellt_kein_name_ein is
port (
A,B,C,D  : IN std_ulogic;
X, Y     : OUT std_ulogic);
end entity mir_faellt_kein_name_ein;
```

Schreiben Sie die *Architecture*, sodass das Modul die folgenden Binärausdrücke berechnet. Es soll möglichst wenig Logik verwendet werden.

$$X = (A \wedge B) \vee C \tag{1}$$

$$Y = (A \wedge B) \vee (C \wedge D) \tag{2}$$

VHDL | Architecture

$$X = (A \wedge B) \vee C, \qquad Y = (A \wedge B) \vee (C \wedge D)$$

VHDL | Architecture

$X = (A \wedge B) \vee C, \qquad Y = (A \wedge B) \vee (C \wedge D)$

```
architecture rtl of mir_faellt_kein_name_ein is


begin

        X <= (A and B) or C;
        Y <= (A and B) or (C and D);

end architecture rtl;
```

VHDL | Architecture

$X = (A \wedge B) \vee C, \qquad Y = (A \wedge B) \vee (C \wedge D)$

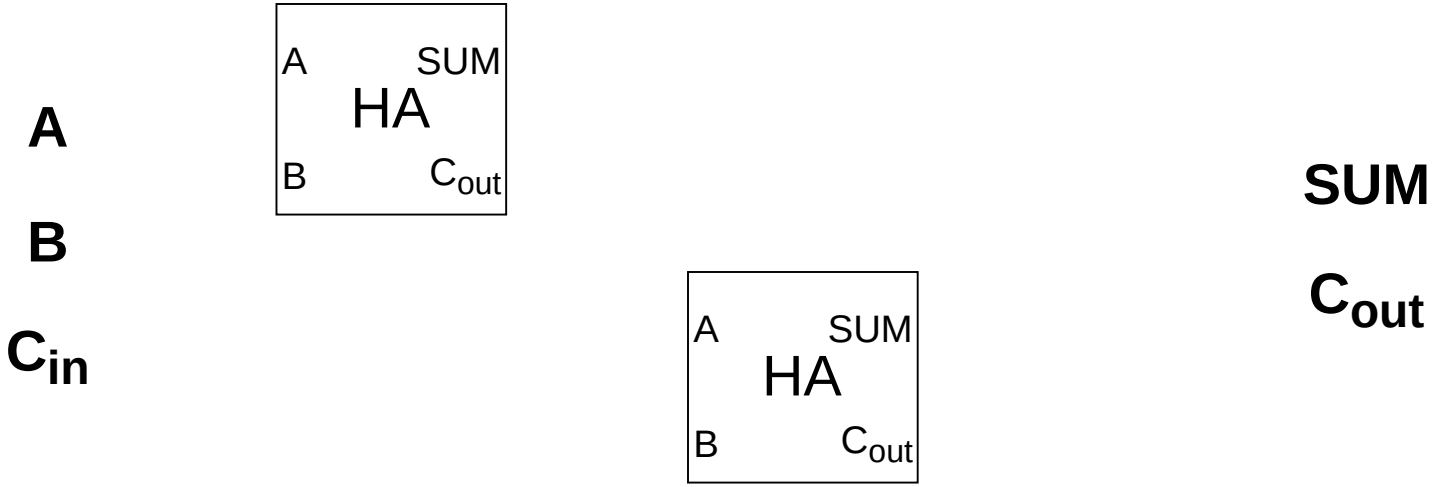
```
architecture rtl of mir_faellt_kein_name_ein is

signal a_and_b_int : std_ulogic;

begin

X <= a_and_b_int or C;
Y <= a_and_b_int or (C and D);
a_and_b_int <= A and B;

end architecture rtl;
```

VHDL | Volladdierer

Folgendes Halbaddierer-Modul ist gegeben. Es soll ein Volladdierer aus zwei Halbaddierer-Modulen gebaut werden.

```
entity HALBADD is
port (
A, B  : IN std_ulogic;
SUM, CO : OUT std_ulogic);
end entity HALBADD;
```

Wie müssen die Halbaddierer-Module verbunden werden, um einen Volladdierer zu bilden?

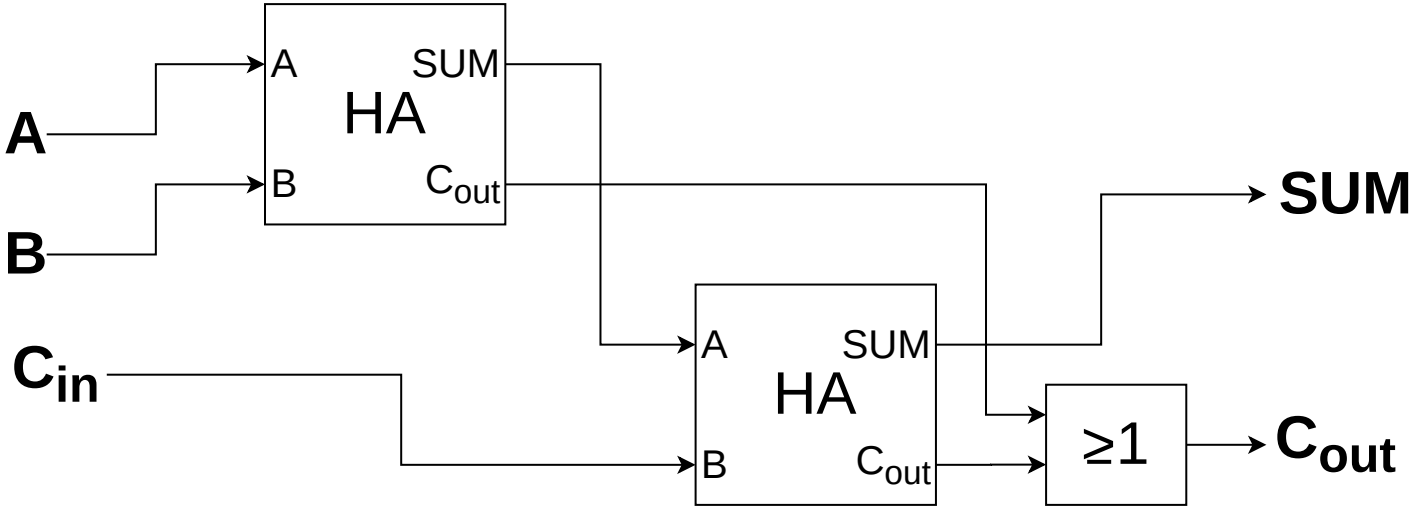


VHDL | Volladdierer

Folgendes Halbaddierer-Modul ist gegeben. Es soll ein Volladdierer aus zwei Halbaddierer-Modulen gebaut werden.

```
entity HALBADD is
port (
A, B  : IN std_ulogic;
SUM, CO : OUT std_ulogic);
end entity HALBADD;
```

Wie müssen die Halbaddierer-Module verbunden werden, um einen Volladdierer zu bilden?



VHDL | Volladdierer

Ergänzen Sie folgenden VHDL-Code, um die Schaltung aus Aufgabenteil a) zu bauen:

VHDL | Volladdierer

Ergänzen Sie folgenden VHDL-Code, um die Schaltung aus Aufgabenteil a) zu bauen:

```
architecture rtl of VOLLADD is
-- EUER CODE VON HIER

-- BIS HIER
```

```
begin
-- EUER CODE VON HIER

-- BIS HIER
end rtl;
```

VHDL | Volladdierer

```
architecture rtl of VOLLADD is
component HALBADD is
port (
  A   : IN std_ulogic;
  B   : IN std_ulogic;
  SUM : OUT std_ulogic;
  CO  : OUT std_ulogic);
end component HALBADD;

signal c0_int, c1_int
         : std_ulogic;
signal sum_int  : std_ulogic;

begin
```

```
HA1_inst: HALBADD
port map(
  A   => A,
  B   => B,
  SUM => sum_int,
  CO  => c0_int);

HA2_inst: HALBADD
port map(
  A   => sum_int,
  B   => C_in,
  SUM => SUM,
  CO  => c1_int);

C_out <= c0_int or c1_int;

end rtl;
```